ESTERIA

Retail-Style Dragonriding / Skyriding

Client + AzerothCore Implementation Guide

Implementation edition: 1.0
Prepared: October 25, 2026
Target: Custom World of Warcraft 3.3.5a client (wowdecomp, build 12340) and AzerothCore WotLK server
Intended implementer: Claude Code, working against locally checked-out source repositories

> **Engineering objective:** Reproduce the *observable gameplay* of Retail Dragonriding / Skyriding as closely as the 3.3.5a client architecture allows: momentum-driven flight, realistic-feeling dive and climb response, abilities, animations, camera, UI, and correctly synchronized multiplayer behavior. Maintain complete compatibility with ordinary movement when Skyriding is inactive.
>
> **Important limitation:** wowdecomp is a reconstruction of the 3.3.5a client, not retail Dragonflight source. Retail's precise physics constants, proprietary implementation details, and all animation behavior are not present in that client. A literal source-level or bit-for-bit 1:1 port is not available from those materials. This guide targets an independently engineered, high-fidelity behavioral recreation using measured references.

TABLE OF CONTENTS

01 Executive build plan....3

02 Repository preparation and baseline....4

03 Phase 0: source reconnaissance....5

04 Architecture and ownership....6

05 Phase 1: shared physics....6

06 Phase 2: wowdecomp integration....8

07 Phases 3-4: AzerothCore and networking....9

08 Phase 5: abilities and resources....11

09 Phase 6: animations, camera, HUD....12

10 Phase 7: retail calibration....13

11 Work breakdown and release gates....14

12 Claude Code implementation prompts....15

13 Testing checklist....17

14 Primary references....18

15 First action to take....18

1. Executive build plan

1.1 What to build

Implement a **dedicated, feature-gated dynamic-flight mode** shared between the modified client and AzerothCore. The core components are:

1. **Portable physics library:** deterministic-ish fixed-step simulation, airborne velocity, gravity, drag, dive/climb response, steering, stall, impulses, and state transitions. The library must not depend on AzerothCore or client renderer types.
2. **Modified client controller:** read mouse and keyboard intent, run immediate client-side prediction, use the existing collision and world position systems, and render correct flight state.
3. **Server movement owner:** authorize eligible mounts, validate ability and input requests, simulate flight authoritatively, relocate players safely, and enforce movement restrictions.
4. **Versioned protocol extension:** negotiate support, send input frames and snapshots, reconcile local state, and interpolate remote flyers. Do not overwrite existing WotLK opcode IDs or packet layouts.
5. **Presentation layer:** per-model animations, procedural banking and pitch, camera options, configurable HUD, and flight-specific abilities.
6. **Reference-based calibration:** instrument flight and compare against defined retail-era maneuvers before claiming parity.

1.2 Non-negotiable invariants

- Normal running, jumping, swimming, falling, taxi paths, transports, vehicles, and Steady Flight behave unchanged when dynamic flight is inactive.
- Exactly **one system owns authoritative player movement** at a time. Never allow vanilla position updates and dynamic-flight simulation to race against each other.
- The server decides who may activate Skyriding, use an ability, change maps, and occupy a position. Never trust client-supplied velocity or position as authority.
- Collision, map bounds, flight restrictions, disconnect cleanup, teleport handling, and dismount-in-air transitions are first-class requirements.
- New code is isolated behind configuration/feature gates with rollback capability.
- Claude must name *verified* symbols, paths, build targets, and test output. Pseudocode in this document is not a claim about repository APIs.

1.3 Choose the behavior profile before calibrating

Behavior axis	Dragonflight-era profile	Current Skyriding profile
Basic movement	Momentum-based dynamic flight	Momentum-based dynamic flight
Ability resource	Vigor / recharge under specific conditions	Shared charges for major movement abilities
Flight-style switch	Era-dependent	Dynamic vs. Steady Flight
Special abilities	Era-specific unlocks and behavior	Current set and revised abilities
Intended scope	Recreation of Dragonflight launch or specified patch	Recreation of a named modern retail patch

Recommendation: implement a physics-independent resource and ability profile. The resource system changed in patch 11.2.7: Blizzard describes removal of Vigor and use of six shared charges for Surge Forward and Skyward Ascent. Pick an exact reference patch when assessing 1:1 parity. Do not mix resource rules from different eras accidentally.

Deliverable for the owner: docs/SKYRIDING_TARGET_SPEC.md recording the reference expansion/patch, desired defaults, supported mounts, target maps, and any intentional Esteria-only deviations.

---

2. Repository preparation and baseline

2.1 Suggested workspace

```
Esteria-Development/
  client/
    wowdecomp/                      # existing checkout, no folder assumptions
  server/
    azerothcore-wotlk/              # existing checkout
  shared/
    skyriding/
      include/
        SkyridingTypes.h
        SkyridingPhysics.h
        SkyridingConfig.h
        SkyridingProtocol.h
      src/
        SkyridingPhysics.cpp
        SkyridingStateMachine.cpp
      tests/
  docs/
    SKYRIDING_TARGET_SPEC.md
    SKYRIDING_ARCHITECTURE.md
    SKYRIDING_CLIENT_MAP.md
    SKYRIDING_SERVER_MAP.md
    SKYRIDING_PHYSICS.md
    SKYRIDING_PROTOCOL.md
    SKYRIDING_ANIMATIONS.md
    SKYRIDING_TESTS.md
```

These are proposed new paths. Claude must preserve established repo structure and build conventions if existing arrangements differ.

2.2 Create a reproducible known-good baseline

1. Commit or tag the current client and server as a pre-Skyriding baseline.
2. Record the active client build, wowdecomp commit, AzerothCore commit, compiler version, game data version, and enabled modules.
3. Build the unmodified reconstructed client and launch it against the existing server.
4. Verify login; movement; conventional flying mounts; flight master taxis; water movement; a vehicle; a transport; map transition; and disconnect/reconnect.
5. Capture server logs, client errors, and a short baseline video.

The published wowdecomp README documents a Visual Studio 2022 x64 build similar to:

```
cmake -S . -B build/x64 -G "Visual Studio 17 2022" -A x64
cmake --build build/x64 --config Release --target WowClient --parallel
```

Confirm commands from **your checked-out revision**. If client assets are modified, check the repository's supported signature-check development option rather than blindly patching the loader.

Exit gate: both original branches build, launch, and pass the movement smoke test. Do not proceed without a stable baseline.

---

3. Phase 0: source reconnaissance with Claude

The decompiled client contains reconstructed classes, partial coverage, and unfamiliar call chains. The first Claude job is an evidence-backed source map, not implementation.

3.1 Client: trace these paths

Area	Questions Claude must answer
Input	Where do mouse pitch, mouse yaw, keyboard movement, jump, and camera freelook enter?
Movement	Where are local position, orientation, velocity/speed, fall, swim, fly and jump updated?
Flags	How are existing flying flags and movement state represented?
Physics/collision	Where does movement call collision, terrain height, liquid, and world-bound queries?
Packets	Which functions construct local movement packets and apply remote movement updates?
Model animation	Which controller chooses rider/mount animation, M2 sequences, pitch, and transform?
UI	Which native APIs and FrameXML files can expose custom state and actions?
Build	Which libraries, source lists, compilation options, and test targets need updates?

Starting search hints, not guaranteed exact function names:

```
rg -n "MOVEMENTFLAG_FLYING|MOVEMENTFLAG_CAN_FLY|MovementInfo" WoW Engine
rg -n "pitch|velocity|MoveForward|Jump|camera|collision" WoW Engine
rg -n "WriteMovement|ReadMovement|Move.*Packet|Send.*Movement" WoW Engine
```

3.2 Server: trace these paths

Begin with the actual source in src/server/game/Handlers/MovementHandler.cpp and related movement/session classes. The upstream AzerothCore file contains functions named WorldSession::VerifyMovementInfo and WorldSession::ProcessMovementInfo; confirm exact signatures and call paths in your checkout.

Trace:

- Packet registration and client/server opcode dispatch.
- Movement info read, verification, position relocation, and broadcast.
- Player movement state, flying permissions, mount aura application/removal.
- Speed checks, anti-cheat, map validation, fall damage, collision, and transport handling.
- Session thread or map update timing, reconnect, teleport, and logout cleanup.
- Existing module hooks and the minimum core patches required for custom movement.

3.3 Required reconnaissance artifacts

For each identified integration point, Claude must record **real file path, real symbol, purpose, caller/callee chain, hazards, and recommended extension location**. A line number alone is not sufficient because it may shift with new commits.

Create:

- SKYRIDING_CLIENT_MAP.md

- SKYRIDING_SERVER_MAP.md
- SKYRIDING_ARCHITECTURE.md

Exit gate: the complete path from physical input to local transform to packet to server relocation to remote client rendering is understood and documented. Unknowns are listed explicitly.

---

4. Architecture and ownership

```
               ESTERIA MODIFIED CLIENT
 Mouse/Keys -> Input Normalizer -> Flight Controller
                       |                 |
                       |          Fixed-step shared simulator
                       |                 |
                       +----> Input log / local prediction
                                         |
                                 Custom input packets
                                         v
               AZEROTHCORE WORLDSERVER
   Capability check -> Authorized input queue
                                         |
                                 Fixed-step shared simulator
                                         |
                             Collision / rules / relocation
                                         |
                    Authoritative snapshots + game events
                           /                         \
                   Local client                 Remote clients
                Reconcile/replay            Buffer/interpolate
                           \                         /
                         Client presentation layer
                 Animations / banking / camera / UI
```

The shared simulator must use explicit world units and documented coordinate conventions. Do not assume the source's handedness or axis order without inspection.

4.1 Movement ownership contract

- Steady/Vanilla: original client and server movement pathways apply as before.
- SkyridingPending: server has authorized the mode; wait for a synchronized beginning tick/state.
- SkyridingActive: flight controller consumes inputs; server simulation owns accepted world position.
- SkyridingEnding: client/server transition to normal grounded movement or ordinary fall, resetting incompatible flags.

All state changes must be idempotent so an out-of-order or duplicated event cannot permanently trap a player in flight mode.

---

5. Phase 1: shared physics implementation

5.1 State design

Split mutually exclusive locomotion and flight submodes from overlapping effects. Example:

```cpp
// Illustrative proposed types. Adapt names to actual project conventions.
enum class FlightLocomotion : uint8_t {
    Grounded, Launching, Airborne, Landing, Disabled
};

enum class AirborneState : uint8_t {
    Glide, Dive, Climb, Stall
};

struct SkyridingState {
    Vec3 position;
    Vec3 velocity;
    float yawRadians;
    float pitchRadians;
    float visualBankRadians;
    FlightLocomotion locomotion;
    AirborneState airborne;
    uint32_t physicsTick;
    float activeBoostSeconds;
};

struct SkyridingInput {
    float targetYawRadians;
    float targetPitchRadians;
    bool launch;
    bool surge;
    bool ascend;
    bool brake;
};
```

Gameplay charges, cooldowns, permissions, and spells belong in separate authoritative systems; the physics library should consume **validated effect commands**, not decide whether a client can afford a cast.

5.2 Physics model: proposed approximation

Represent flight with world-space velocity $\mathbf{v}$ and a direction derived from yaw/pitch. Use a game-feel model rather than an overly elaborate aerodynamic simulator:

```
forward_speed_dot = approved_boost_accel
                  - effective_gravity * sin(flight_path_pitch)
                  - (linear_drag * speed + quadratic_drag * speed^2)

vertical_speed = forward_speed * sin(flight_path_pitch) - sink_rate(speed)
position_next = ResolveCollision(position, candidate_displacement)
```

These are conceptual relationships only. A real implementation must prevent double-applying gravity when world velocity already contains a vertical component. Choose **one coherent dynamic formulation**, document it, and test energy behavior. In particular, do not separately integrate $\mathrm{vertical_speed}$ and then apply an unrelated gravity acceleration to the same state.

Essential responses

Maneuver	Observable behavior
Dive	Steeper descent increases speed until configured bounds
Climb	Consumes stored momentum and exchanges speed for altitude
Glide	Maintains forward momentum but ordinarily sinks over time
Turn	Smoothly rotates travel direction; high-speed turns feel wider
Stall	Insufficient speed limits climb and leads to altitude loss
Surge	Applies a time-bounded forward acceleration or impulse
Ascend	Applies an upward impulse with appropriate forward behavior
Land	Resolves ground contact and transitions into ordinary locomotion

Do not replace velocity each frame with $\text{forwardDirection} * \text{fixedSpeed}$: that produces flight *steering*, not momentum-driven flight.

5.3 Fixed step and determinism

Suggested development starting point: fixed **50 Hz** physics ($dt = 0.02$ s), with a bounded accumulation loop so lag spikes do not trigger thousands of steps. This is a design choice, not a known retail setting.

```
constexpr double kStepSeconds = 1.0 / 50.0;
accumulator = std::min(accumulator + frameSeconds, 0.25);
while (accumulator >= kStepSeconds) {
    SimulateFixedStep(state, validatedInput, kStepSeconds);
    accumulator -= kStepSeconds;
}
```

For server and client parity, establish floating-point and compiler behavior assumptions. Exact cross-platform bitwise determinism is not guaranteed by shared C++ alone. Make the reconciliation protocol tolerant of small differences.

5.4 Parameter profiles

Provide a centralized, validated configuration with units and ranges. Example field names:

```
[Skyriding.Dragonflight]
SimulationHz = 50
GravityScale = <measured-or-provisional>
DragLinear = <measured-or-provisional>
DragQuadratic = <measured-or-provisional>
MinGlideSpeed = <measured-or-provisional>
StallSpeed = <measured-or-provisional>
MaxAirSpeed = <measured-or-provisional>
DiveResponse = <measured-or-provisional>
TurnResponse = <measured-or-provisional>
SurgeImpulse = <measured-or-provisional>
AscentImpulse = <measured-or-provisional>
```

The placeholders are intentional. Claude must choose safe *clearly labeled provisional* test values before running the simulator. Do not represent invented constants as Blizzard's numbers.

5.5 Physics exit criteria

- Dive gains speed; climb loses speed and gains altitude; level glide loses altitude at a controlled rate.
- Flight persists without forward input; the mount does not instantly stop in midair.
- At low speed, stall and recovery behave predictably.
- Given the same fixed-step input sequence, runs are numerically stable and sufficiently reproducible.
- Speed, acceleration, and pitch stay finite, bounded, and free of NaN/Inf under malformed inputs.
- The standalone library builds and tests without invoking the game client or server.

---

6. Phase 2: integrate with wowdecomp

6.1 Feature-gated control handoff

At the verified player movement update site, branch into a new controller only when the server-authorized local player is in the active dynamic-flight mode:

```
// Integration sketch, NOT a wowdecomp API excerpt.
void UpdatePlayerMovement(double frameSeconds) {
    if (IsAuthorizedLocalSkyridingActive()) {
        SkyridingInput input = ReadSkyridingControls();
        PredictSkyriding(input, frameSeconds);
        ApplySimulatedMovementThroughExistingWorldPath();
        return;
    }
    UpdateOriginalPlayerMovement(frameSeconds);
}
```

Preserve existing systems such as terrain/geometry collision, object update, map load, dynamic objects, UI notifications, and packet publication. Reuse the appropriate engine abstractions rather than writing coordinates directly to unrelated fields.

6.2 Controls and camera

Recommended mappings, configurable per character:

Control	Action
Takeoff input	Request authorized launch / initial impulse
Right-mouse steer	Change desired yaw and pitch
Freelook	Move camera without necessarily altering trajectory
Surge Forward binding	Request acceleration ability
Skyward Ascent binding	Request upward ability
Aerial Halt binding	Request deceleration ability
Pitch accessibility bindings	Alternate controls for users who cannot rely on mouse pitch

Do not hardcode a double-tap-space rule as the only possible launch control. It can be a configurable retail-inspired default.

Use smoothing for directional steering. One useful coefficient is $\text{alpha} = 1 - \exp(-\text{response} * \text{dt})$. Steering must preserve sensible velocity and cannot secretly inject arbitrary energy into the system.

6.3 Debug instrumentation

A client development overlay should show: locomotion and airborne state; speed; vertical speed; position; yaw; pitch; bank; simulation tick; acknowledged server tick; pending inputs; prediction correction distance; and last ability event.

Exit gate: one chosen mount can take off, dive, climb, turn, stall, land, and transition back to normal movement *locally*. Other game movement works unchanged.

---

7. Phase 3 and 4: AzerothCore and networking

7.1 Server eligibility and transition rules

Build a server-side registry keyed by mount spell or another stable identifier. Server determines: availability, flight style, legal map/zone, relevant auras, player status, ability permissions, and permissible world movement.

Suggested migration, adapted to Esteria's database naming conventions:

```
CREATE TABLE esteria_skyriding_mounts (
    spell_id INT UNSIGNED NOT NULL PRIMARY KEY,
    enabled TINYINT UNSIGNED NOT NULL DEFAULT 1,
    physics_profile VARCHAR(64) NOT NULL,
    animation_profile VARCHAR(64) NOT NULL
);
```

The module should hook eligible mount aura transitions but may require a small patch to core movement and opcode dispatch. Keep all core patches documented and confined to well-defined handoff points.

7.2 Proposed protocol

These are **new Esteria message concepts**, not existing Blizzard opcode names:

Direction	Message	Function
Server -> client	SMSG_ESTERIA_SKYRIDE_BEGIN	Authorize mode and provide initial tick/state
Client -> server	CMSG_ESTERIA_SKYRIDE_INPUT	Send sequence-numbered normalized intent
Server -> client	SMSG_ESTERIA_SKYRIDE_STATE	Broadcast validated state snapshot
Server -> client	SMSG_ESTERIA_SKYRIDE_EVENT	Signal approved ability/state transitions
Server -> client	SMSG_ESTERIA_SKYRIDE_END	End mode and specify transition behavior

Protocol rules: register unused IDs only after auditing the relevant dispatch tables; version the feature; define exact field widths, endianness, bounds, enum values, sequence wrap handling, and failure behavior. Do not silently alter standard movement packets.

Example payload sketches:

```
struct SkyridingInputWire {
    uint32_t sequence;
    uint32_t clientTick;
    float requestedYaw;
    float requestedPitch;
    uint16_t buttonBits;
};

struct SkyridingSnapshotWire {
    uint64_t playerGuid;
    uint32_t serverTick;
    uint32_t acknowledgedSequence;
    Vec3 position;
    Vec3 velocity;
    float yaw;
    float pitch;
    uint8_t locomotion;
};
```

Do not write these C++ structs raw onto the wire: compilers can introduce padding, layout differences, and unsupported endian assumptions. Use explicit serializer/deserializer functions.

7.3 Correct authority model

The client sends **intent**, not trusted positions or speeds. The server:

1. Authenticates the session and negotiates capability.
2. Checks active mount, zone, player state, sequence order, input rates, and payload bounds.
3. Queues inputs for a bounded simulation time window.
4. Advances authoritative flight state at a fixed step.
5. Sweeps/resolves collision and applies world constraints and ability rules.
6. Updates AzerothCore's real world position using safe relocation hooks.
7. Emits state snapshots and approved events for relevant observers.

Do not disable normal anti-cheat globally. Route dynamic flight through **mode-specific** speed, acceleration, displacement, flight-permission, and collision checks while retaining all checks for normal movement.

7.4 Prediction, reconciliation, and replication

Local player: predict immediately and store input frames. On a server snapshot, reset to the accepted state at the acknowledged sequence, replay unacknowledged input, and smooth only the rendered discrepancy. Never smooth a collision correction through solid geometry.

Remote players: buffer server snapshots, interpolate with a slight intentional visual delay, and restrict extrapolation. The server retains responsibility for range, visibility, map transfers, and combat-relevant position.

Suggested initial operating points: server simulation at 50 Hz and snapshots at 10-20 Hz. These are test hypotheses, not retail network specifications. Measure bandwidth, server tick cost, and latency before optimization.

7.5 Mandatory edge cases

- Collision with steep terrain, towers, ceilings, and map borders at maximum speed.
- Dismounting midair and transitioning to ordinary fall/fall damage.
- Entering a no-fly zone or battleground.
- Death, combat restrictions, stun, fear, root, or forced movement.
- Teleport, hearthstone, transport boarding, vehicle boarding, and map transfers.
- Reconnect and logout during flight.
- Two players flying in the same area; one remote player dismounting.
- PlayerBots or server-controlled characters with incompatible legacy movement.
- Packet reorder, duplication, loss, oversized payloads, and malicious input rates.
- Unmodified/unsupported client version: explicit feature refusal or defined fallback.

Exit gate: two modified clients can see and interact with the same server-authoritative flying characters without rubberbanding under ordinary conditions. Network fault tests do not crash or desynchronize the worldserver.

---

8. Phase 5: abilities, charges and progression

Implement ability logic on the **server** as validated commands that affect shared physics, with spell/UI presentation layered on top. Avoid pretending that a repeating speed aura is actual dynamic flight.

Ability / feature	Engine effect	Authority concerns
Takeoff / Lift Off	Ground -> launch; initial upward/forward impulse	Ground contact, eligibility, lockouts
Surge Forward	Time-bounded forward acceleration/impulse	Resource, cooldown, anti-spam
Skyward Ascent	Upward impulse and revised pitch/momentum	Resource, collision, ceiling
Whirling Surge	Configured powerful acceleration with animation	Cooldown, stacking, max speed
Aerial Halt	Controlled air-brake and associated transition	Velocity bounds, legal states
Thrill of the Skies	Detect sustained high speed for resource effect	Server speed state, patch profile
Ground Skimming	Near-surface rule when enabled by chosen patch	Terrain query, surface validity
Bronze Timelock / Rewind	Save and validate restore checkpoint	Map, teleport and exploit constraints

Begin with Takeoff, Surge Forward, and Skyward Ascent. Implement the others after flight and replication are stable. Some ability names and rules change between patches; do not apply every ability to every target profile automatically.

Separate ISkyridingResourcePolicy-style gameplay interfaces from the physics library. One policy can model original Vigor, another the later six-charge system. Data-drive the availability of progression and visual indicators.

Exit gate: all enabled abilities produce identical accepted gameplay behavior on both client prediction and server snapshots; no cast bypasses authorization or charge/cooldown checks.

---

9. Phase 6: animation, camera, and HUD

9.1 Mount animation controller

Render state should be generated from speed, pitch, yaw rate, bank, boost event, and grounded/airborne submode. Map to the best available M2 sequence on a **per-model basis**.

State	Expected presentation
Launch	Strong flap, body rise, wing transition
Glide	Sustained extended-wing pose/loop
Dive	Nose-down posture, optional swept-wing pose
Climb	Nose-up posture and heavier wing action
Turn	Smooth roll/bank into turn
Surge	Acceleration wingbeat and speed feedback
Stall	Unstable low-speed pose and recovery
Land	Contact-aware aerial-to-ground transition

Support three profile types: Native for imported models with appropriate animations, Adapted for blending WotLK loops and procedural transforms, and Fallback for simple mounts missing specialized clips.

Critical separation: visual bank is a presentation transform; it should not rotate the collision capsule or arbitrarily change the server flight path.

9.2 Camera options

- Modest optional FOV increase at high speed.
- Smooth follow for pitch and fast turns.
- Free camera look independent of steering when chosen.
- Configurable motion intensity, screen effects, and camera banking.
- Keyboard steering accessibility and sensitivity curves.

9.3 FrameXML / Lua UI

Keep UI presentation in FrameXML/Lua, with a **new native API** providing approved flight state and client-predicted display values. Names below are *proposals*, not retail APIs:

```lua
local state = GetEsteriaSkyridingState()
local speed = state.speed
local charges = state.charges
local locomotion = state.locomotion
```

The server remains authoritative for charge counts and cooldowns. Use UI events rather than busy-looping Lua for physics. Provide a profile-sensitive Vigor or shared-charge bar and accessible keybinding hints.

Exit gate: selected test mount visually banks, takes off, glides, dives, climbs, boosts, and lands; camera and UI settings survive reload; disabling visual extras does not affect authoritative physics.

---

10. Phase 7: calibration to retail reference behavior

10.1 Measurement protocol

Use one clearly identified target retail patch. For each maneuver, record known initial mount, ability/progression state, starting point/height, input trace, elapsed times, and reference observations. Videos are valuable, but they alone rarely expose authoritative world-unit speed or internal force constants.

Test	Calibration output
Ground takeoff	Height-vs-time and launch animation latency
Straight dive	Speed-vs-time, terminal speed approach
Fixed-angle climb	Speed spent per altitude gained
Level glide	Altitude lost per second at several speeds
Stall and recovery	Stall threshold, descent and recovery delay
Low/high-speed turns	Turn radius and yaw response
Surge Forward	Speed gain profile and duration
Skyward Ascent	Height gained and speed after impulse
Landing	Contact resolution and control handoff latency
Multiplayer	Position error, interpolation lag, server corrections

Instrument Esteria with a CSV or structured log containing: timestamp; input sequence; server/client tick; position; velocity; pitch; yaw; bank; locomotion state; active effects; snapshot latency; and correction magnitude.

Do **not** publish fabricated measurements as retail constants. Record whether each number is observed, estimated, or tuning-only.

10.2 Suggested test environments

Test empty open terrain first, then canyons, steep vertical surfaces, caves, urban geometry, water, roofs, and high-altitude maps. Re-run all maneuvers at variable client FPS and simulated latency. Verify maximum-speed collision safety.

10.3 Acceptance rubric

- **Movement fidelity:** dive/climb/glide/stall/turn feel and curves match chosen references within documented tolerance.
- **Network fidelity:** local control is responsive; remote viewers see coherent motion; correction frequency and magnitude remain bounded.
- **Gameplay fidelity:** chosen-era abilities, resources, restrictions and progression behave as specified.
- **Visual fidelity:** chosen mounts play appropriate motion and blend correctly with camera/UI.
- **Regression safety:** ordinary movement, vehicles, taxis, maps and mounts remain functional.

Declare high-fidelity recreation, not retail-identical, until sufficient measured evidence supports narrower claims.

---

11. Work breakdown and release gates

Milestone	Claude deliverable	Hard pass condition
M0 - Audit	Verified movement source maps, architecture, version spec	No source files changed; actual symbols cited
M1 - Shared simulator	Portable library, tunable profiles, tests	Deterministic-ish fixed-step replay and all core movement tests pass
M2 - Local client	Controlled handoff, debug overlay, collision path	Flight works locally; vanilla movement unaffected
M3 - Worldserver	Mode eligibility, mount registry, lifecycle	No unauthorized activation; safe exit/teleport/dismount
M4 - Networking	Versioned protocol, snapshots, prediction	2 clients pass latency and edge-case tests
M5 - Gameplay	Abilities, charge policies, cooldowns	All effects authorized and replicated
M6 - Presentation	Animations, camera, FrameXML	High-quality supported mounts and fallback profile
M7 - Calibration	Reference maneuvers and test report	Documented differences, no severe movement regressions

For every milestone, require Claude to run build/tests and provide: edited files, why each was edited, test command/result, remaining issues, and precise rollback steps.

12. Ready-to-use Claude Code prompts

Use these prompts **one milestone at a time**. Paste the entire prompt into Claude Code, while giving it access to the actual checkout. Never ask it to claim a build passed without running it.

Prompt A: Repository audit (start here)

```
You are implementing Esteria's retail-style Skyriding on:
- A rebuildable 3.3.5a wowdecomp client
- An AzerothCore WotLK server

Read Esteria_Skyriding_Implementation_Guide.md.
PHASE M0: RECONNAISSANCE ONLY. DO NOT EDIT SOURCE.

Trace the complete client movement path: keyboard/mouse input,
camera orientation, locomotion and fly flags, predicted position,
collision/terrain, movement packets, mount animation and FrameXML.
Trace server opcode dispatch, movement verification/relocation,
mount auras/flight permissions, movement speed checks, anti-cheat,
map and vehicle/transport transitions, and remote broadcasts.

For EVERY integration point provide a verified path, symbol,
call chain, known limitations, and safe extension recommendation.
Do not invent symbols, APIs, packet IDs or file names.

Create docs/SKYRIDING_CLIENT_MAP.md,
docs/SKYRIDING_SERVER_MAP.md and
docs/SKYRIDING_ARCHITECTURE.md.

Conclude with a build-tested baseline status and a list of
unresolved technical questions. No implementation until M0 passes.
```

Prompt B: Shared physics core

```
PHASE M1 ONLY. Read the guide and approved M0 source maps.
Create a standalone portable C++ Skyriding simulator.
Implement fixed-step state, velocity persistence, diving,
climbing, gliding sink, turn inertia, stalls/recovery, launch,
land, validated impulse hooks, profiles and numeric safeguards.

Keep gameplay charges and WoW-specific types OUT of the core.
Treat all unknown retail constants as provisional tunables.
Document unit conventions and coordinate axes.

Write tests for dive, climb, glide, no-input momentum, turns,
stall/recovery, impulse effects, frame-rate independence,
bounds and bad input. Run them. Generate SKYRIDING_PHYSICS.md.
Do not edit client or server in this phase.
```

Prompt C: Client integration

```
PHASE M2 ONLY. Use verified source map; re-check current symbols.
Introduce an authorized, feature-gated Skyriding controller
in wowdecomp. Route local input into the shared physics step;
integrate with the actual engine collision and world-object
relocation path. Preserve vanilla behavior outside this mode.
Implement controllable steering, freelook separation and a
Skyriding debug overlay.

Do not guess function names. Do not touch networking yet.
Build WowClient. Give changed paths, tests, errors and rollback.
```

Prompt D: Server authority and lifecycle

```
PHASE M3 ONLY. Implement AzerothCore authorization, mount
eligibility registry, mode state machine, map/zone restrictions,
safe relocation and explicit cleanup for dismount in air,
teleport, death, logout, reconnect and vehicle transitions.

Prefer module hooks; document unavoidable minimum core changes.
Keep ordinary flying/walking and security checks unchanged.
Write automated checks where possible; run the server build.
Do not register custom opcodes until the protocol phase.
```

Prompt E: Protocol and multiplayer

```
PHASE M4 ONLY. Read SKYRIDING_PROTOCOL.md or create it first.
Design and implement a versioned capability handshake, unique
approved opcode assignments, explicit packet serialization,
bounded client input frames, server authoritative simulation,
snapshots, local prediction/replay and remote interpolation.

Client sends intent, not accepted position. Avoid simultaneous
vanilla/new movement ownership. Never disable anti-cheat
system-wide. Validate sequence, input rate, numeric ranges,
map position, speed, flight restrictions and collisions.

Test with 2 clients, packet loss, latency, dismount, teleport,
reconnect and unsupported client. Build both targets.
Record packet structure and test results precisely.
```

Prompt F: Abilities and resources

```
PHASE M5 ONLY. Implement Lift Off, Surge Forward and Skyward
Ascent using server-approved physics commands. Add profile-
selectable Dragonflight Vigor and modern shared-charge policies.
Integrate cooldown/resource state with prediction and snapshots.
Do not simulate physics by spamming speed auras.

After these pass, add patch-appropriate Aerial Halt, Whirling
Surge and optional progression-dependent features.
Write permission, resource, cooldown and replication tests.
```

Prompt G: Presentation and controls

```
PHASE M6 ONLY. Build a per-mount animation profile system
(Native / Adapted / Fallback). Add render-only pitch/bank,
launch, glide, dive, climb, boost, stall and landing animation
selection based on actual model animation availability.

Create camera options (FOV, smoothing, effects, freelook),
accessible pitch controls and a FrameXML flight HUD.
Expose state through new documented native Lua APIs.
Never let visual banking alter collision or simulation.
Verify all chosen test mounts and normal character rendering.
```

Prompt H: Fidelity and release testing

```
PHASE M7 ONLY. Construct a repeatable flight benchmark suite
using our declared retail target patch. Measure takeoff,
dive, climb, level glide, stall, turn radius, surge, ascent,
landing, collision and network correction.

Label all constants observed, estimated or tuning-only.
Export quantitative comparison tables and flight telemetry.
Run non-Skyriding movement regressions, multiple FPS and
latency profiles, and high-speed collision tests.
Do not claim 1:1 parity unless evidence justifies it.
Provide release blockers, unresolved differences and rollback.
```

13. Testing checklist

Source/build

- ☐ Exact client and server commits recorded
- ☐ Both baselines build and launch
- ☐ Client and server source maps reviewed
- ☐ All custom opcodes audited for collisions
- ☐ Build integration does not break existing targets

Flight physics

- ☐ Launch
- ☐ No-input momentum
- ☐ Dive acceleration
- ☐ Climb and altitude exchange
- ☐ Glide sink
- ☐ Stall and recovery
- ☐ Speed-dependent turning
- ☐ Maximum speed and bad-input bounds
- ☐ World collision at high speed
- ☐ Clean landing

Gameplay/network

- ☐ Server controls eligibility and abilities
- ☐ Unsupported clients do not crash
- ☐ 2-player synchronization
- ☐ Remote pitch/bank interpolation
- ☐ Packet loss, reorder, duplicate packet handling
- ☐ Teleport, map change, transport and vehicle
- ☐ Death, logout/reconnect, midair dismount
- ☐ Ordinary movement unchanged

Visual/UX

- ☐ Takeoff/flight/landing animations
- ☐ Native/adapted/fallback mount profile
- ☐ Render banking is independent of collision
- ☐ Camera settings configurable
- ☐ HUD matches selected resource profile
- ☐ UI receives server-authoritative resources
- ☐ Debug overlay can be disabled for release

14. Primary references

These links are entry points for Claude's verification. Always inspect the checked-out repository and target retail patch directly; the links are not a substitute for source reconnaissance.

1. **wowdecomp reconstruction:** https://github.com/SatyPardus/wowdecomp
2. **AzerothCore MovementHandler:** https://github.com/azerothcore/azerothcore-wotlk/blob/master/src/server/game/Handlers/MovementHandler.cpp
3. **AzerothCore WorldSession declarations:** https://github.com/azerothcore/azerothcore-wotlk/blob/master/src/server/game/Server/WorldSession.h
4. **AzerothCore MovementInfo reference:** https://www.azerothcore.org/doxygen/d2/dd1/structMovementInfo.html
5. **Blizzard Skyriding / shared charges update:** https://news.blizzard.com/en-us/article/24263354/welcome-home-a-returning-player-s-guide
6. **Skyriding ability history and patch differences:** https://www.wowhead.com/guide/skyriding/how-to-ride-your-dragon
7. **M2 animation sequence reference:** https://wowdev.wiki/M2/AnimationList

IP and distribution: A reconstructed WotLK client is not an automatically licensed copy of modern retail implementation code. Avoid distributing proprietary code or assets without the required rights, and assess Blizzard's applicable terms before releasing a patched client.

---

15. First action to take

1. Place this Markdown file beside the Esteria workspace so Claude can read it.
2. Start Claude Code in the workspace containing both repositories.
3. Paste **Prompt A** from section 12.
4. Review the three M0 source maps before allowing any movement edits.
5. Build M1 shared physics in isolation and proceed one acceptance gate at a time.

Definition of success: Esteria delivers flight that behaves, looks, and synchronizes like the selected retail Skyriding era, while the rest of WoW 3.3.5a remains intact.